

Practical - 4

AIM:

Find the Maximum Temperature Recorded for Each Year of NCDC Data using MapReduce

PROBLEM DEFINITION:

Scenario: A meteorological research organization maintains historical weather records collected from various weather stations worldwide. The organization uses Hadoop to analyze large climate datasets and study long-term temperature trends. As a Data Analytics Engineer, you are required to process the National Climatic Data Center (NCDC) dataset and identify the maximum temperature recorded for each year using MapReduce.

OUTPUT:

Task 1: Upload the NCDC weather dataset to HDFS and examine the structure of the weather records.

```
start Practical

from google.colab import files

uploaded = files.upload()

Choose Files 1901.txt
1901.txt(text/plain) - 888190 bytes, last modified: 6/14/2008 - 100% done
Saving 1901.txt to 1901.txt

from google.colab import files

uploaded = files.upload()

Choose Files 1902.txt
1902.txt(text/plain) - 888978 bytes, last modified: 6/14/2008 - 100% done
Saving 1902.txt to 1902.txt

!ls -lh

total 931M
-rw-r--r--  1 root root 868K Aug 23 08:03 1901.txt
-rw-r--r--  1 root root 869K Aug 23 08:04 1902.txt
drwxr-xr-x 11 1024 1024 4.0K Aug 23 07:42 hadoop-3.4.1
-rw-r--r--  1 root root 929M Oct 10 2024 hadoop-3.4.1.tar.gz
drwxr-xr-x  3 root root 4.0K Aug 23 07:42 hadoopdata
drwxr-xr-x  1 root root 4.0K Aug 20 13:29 sample_data
-rw-r--r--  1 root root  75 Aug 23 07:43 sample.txt
```

Task 2: Develop and implement the Mapper program to extract the year and temperature values from each record and generate intermediate key-value pairs.

Mapper.py

TASK 8 — Create the NCDC Mapper

```
%%writefile mapper.py

import sys

for line in sys.stdin:
    line = line.strip()

    # Ignore empty or malformed records
    if len(line) < 93:
        continue

    try:
        # NCDC fixed-width fields
        year = line[15:19]
        temperature = line[87:92]

        # Ignore missing temperature values
        if temperature in ("+9999", "-9999"):
            continue

        temperature = int(temperature)

        # Mapper output: year <TAB> temperature
        print(f"{year}\t{temperature}")

    except ValueError:
        continue

... Writing mapper.py
```

Task 3: Develop and implement the Reducer program to determine the maximum temperature recorded for each year.

Reducer.py

TASK 12 — Create the Reducer

```
%%writefile reducer.py

import sys

current_year = None
max_temperature = None

for line in sys.stdin:
    line = line.strip()

    if not line:
        continue

    try:
        year, temperature = line.split("\t")
        temperature = int(temperature)

    except ValueError:
        continue

    # If a new year is found
    if current_year != year:

        # Print maximum of previous year
        if current_year is not None:
            print(f"{current_year}\t{max_temperature}")

        current_year = year
        max_temperature = temperature

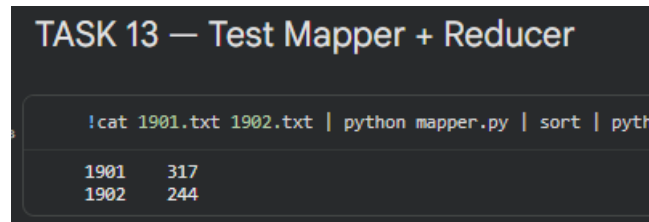
    else:

        # Update maximum temperature
        if temperature > max_temperature:
            max_temperature = temperature

# Print the last year
if current_year is not None:
    print(f"{current_year}\t{max_temperature}")

... Writing reducer.py
```

Task 4: Execute the MapReduce job on Hadoop and store the results in HDFS. Verify the correctness of the output generated.

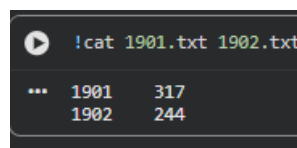


```
TASK 13 — Test Mapper + Reducer

!cat 1901.txt 1902.txt | python mapper.py | sort | pyth

1901    317
1902    244
```

Task 5: Analyze the yearly maximum temperature trends and identify the hottest year in the dataset



```
!cat 1901.txt 1902.txt

... 1901    317
     1902    244
```

KEY QUESTIONS:**1. How is weather data parsed and processed using MapReduce?**

The weather data is stored in HDFS and read by the Mapper. The Mapper takes the required information from each record, mainly the year and temperature. Hadoop then groups the data according to the year, and the Reducer finds the highest temperature for each year.

2. What key-value pairs are generated by the Mapper?

The Mapper generates the year as the key and temperature as the value.

1901 -194

1901 -156

1901 -150

So, the key is **year** and the value is **temperature**.

3. How does the reducer identify the maximum temperature for a given year?

The Reducer receives all temperature values for the same year and compares them. It keeps the largest value and gives that value as the maximum temperature for that year.

For example:

1901 -194

1901 -156

1901 317

The result will be:

1901 317

4. What challenges arise when processing large climate datasets?

Large climate datasets contain a very large number of records. Some records may also contain missing or incorrect values. Processing such data requires proper data handling, storage, and efficient processing of the records.

5. How does Hadoop improve scalability and performance for weather data analytics?

Hadoop divides large datasets into smaller parts and processes them in parallel. HDFS provides distributed storage, while MapReduce performs distributed processing. Therefore, Hadoop can handle large weather datasets more efficiently than processing everything on a single machine.

CONCLUSION:

In this practice, the NCDC weather data was successfully processed using Hadoop MapReduce. The Mapper extracted the year and temperature from the records, and the Reducer calculated the maximum temperature for each year. The final results were stored in HDFS and verified successfully. This practice helped demonstrate how Hadoop can be used to process large weather datasets using distributed processing.

GITHUB : https://github.com/Devsavani12/BIG-DATA_SHP